

9강. API 게이트웨이



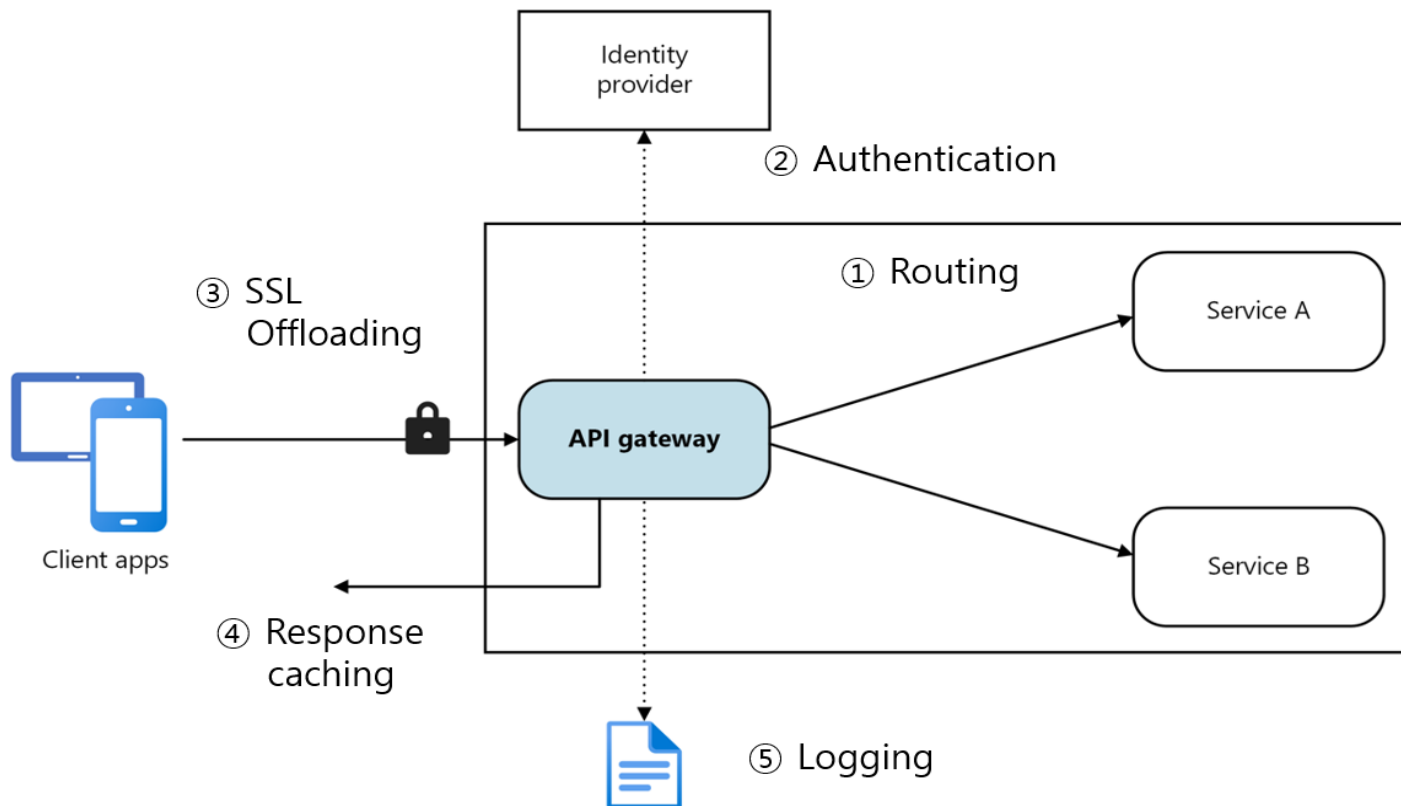
한국공학대학교
TECH UNIVERSITY OF KOREA

이영곤



기본 개념

- ✓ API Gateway는 클라이언트와 마이크로서비스 사이의 단일 진입점
- ✓ 모든 요청을 수신하여 적절한 서비스로 라우팅
- ✓ 인증, 로깅, 모니터링, 트래픽 제어 등의 공통 기능 처리



G/W 없이 서비스를 다이렉트 접근시 문제점

- ✓ 클라이언트가 여러 서비스 직접 호출
- ✓ 서비스 변경 시 클라이언트 수정 필요
- ✓ 인증/로깅 중복 구현
- ✓ 네트워크 복잡도 증가

Gateway 도입 효과

- ✓ 단일 진입점 제공
- ✓ 공통 기능 중앙화
- ✓ 클라이언트 단순화
- ✓ 보안 강화

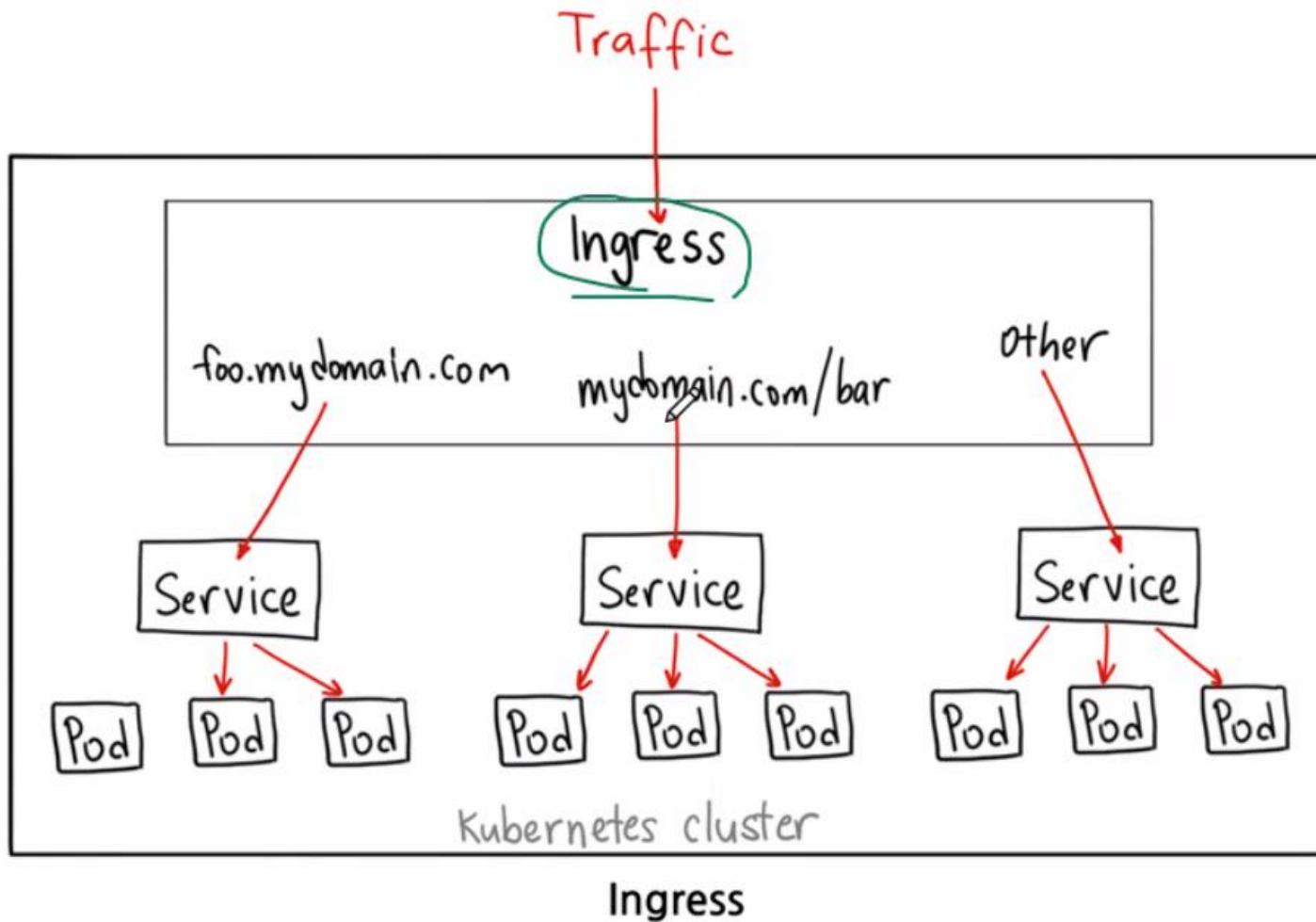
1. 라우팅 (Routing)

- ✓ Path, Host, Header 기반 라우팅
- ✓ 정적 라우팅 vs 동적 라우팅 (Service Discovery 연계)
- ✓ Rewrite, PrefixStrip, Path 변환
- ✓ Canary / Blue-Green 라우팅

라우팅 방식: Path 기반 라우팅

- ✓ URL 경로(Path)에 따라 다른 서비스로 전달하는 방식
 - `/api/orders/**` → `order-service`
 - `/api/users/**` → `user-service`
 - `/static/**` → `static-server`
 - `GET /api/orders/100` ⇒ `order-service` 로 전달
- ✓ 특징
 - 가장 많이 사용
 - REST API 구조와 잘 맞음
 - API Gateway, Ingress, Service Mesh 모두 지원

라우팅 방식: Path 기반 라우팅



라우팅 방식: Path 기반 라우팅

```
rules:
- http:
  paths:
  - path: /orders
    pathType: Prefix
    backend:
      service:
        name: order-service
        port:
          number: 8080
  - path: /users
    pathType: Prefix
    backend:
      service:
        name: user-service
        port:
          number: 8080
```

라우팅 방식: Host 기반 라우팅

✓ 도메인(hostname)에 따라 서비스 분기

✓ 예시:

- api.company.com → api-service
- admin.company.com → admin-service
- shop.company.com → shop-service

✓ 특징

- 서비스별 도메인 분리 가능
- SaaS 멀티테넌트 구조에서 자주 사용
- HTTPS 인증서 관리와 연결됨

EXAMPLE

```
tls:  
- hosts:  
  - "*.example.com"  
secretName: wildcard-cert
```


라우팅 방식: Host 기반 라우팅

```
rules:
- host: api.company.com
  http:
    paths:
    - path: /
      backend:
        service:
          name: api-service
- host: admin.company.com
  http:
    paths:
    - path: /
      backend:
        service:
          name: admin-service
```

라우팅 방식: Header 기반 라우팅

- ✓ HTTP Header 값을 보고 라우팅
 - 예: X-Version: v2
- ✓ A/B 테스트
 - 일부 사용자만 새 버전 사용
- ✓ 내부 관리자 기능
 - X-Admin: true
- ✓ 모바일/웹 분리
 - User-Agent: Mobile

```
http:
- match:
  - headers:
    x-version:
      exact: v2
  route:
  - destination:
    host: product-service
    subset: v2
```

라우팅 방식: 정적 라우팅

- ✓ 라우팅 대상이 고정
- ✓ 특징
 - 단순
 - 빠름
 - 운영 중 변경 어려움
 - 서버 IP 변경 시 수정 필요

```
route:  
  host: 10.0.0.10
```

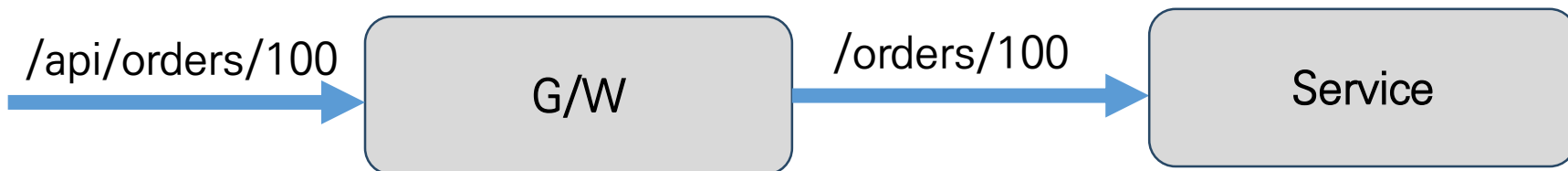
```
backend:  
  service: order-service
```

라우팅 방식: 동적 라우팅

- ✓ Service Discovery 와 연계하여 실시간으로 대상 변경
- ✓ Pod가 죽으면 자동 제외
- ✓ 새 Pod 생성 시 자동 추가
- ✓ Service ⇒ Endpoints 자동 관리 ⇒ Ingress / Istio가 실시간 참조
- ✓ 장점:
 - 오토스케일링 지원
 - 장애 자동 대응
 - 무중단 배포 가능

Rewrite

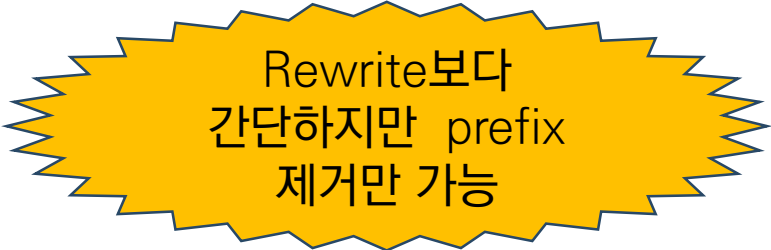
- ✓ 외부에서 요청된 Path를 내부 상황에 맞게 변경
- ✓ 예:
 - 외부 요청: /api/orders/100
 - 내부 전달: /orders/100
- ✓ Nginx 예시:
 - `rewrite ^/api/(.*)$ /$1 break;`



PrefixStrip

- ✓ url 중 앞부분 제거
- ✓ 예:
 - 외부 요청: /api/orders/100
 - 내부 전달: /orders/100
- ✓ 사례:

```
middlewares:  
  strip-api:  
    stripPrefix:  
      prefixes:  
        - /api
```



Rewrite보다
간단하지만 prefix
제거만 가능

Rewrite와 PrefixStrip 사용 이유

- ✓ 외부 API 표준화
 - 외부: /api/v1/*
 - 내부: /orders/*
- ✓ 레거시 연동
 - 구형 시스템: /legacy/*
 - 신규 시스템: /modern/*
- ✓ 마이크로서비스 통합
 - 여러 서비스가 서로 다른 path 체계 사용 시 통합 가능

Canary 라우팅

✓ 새 버전을 일부 사용자에게만 점진 배포

✓ 개념

- 90% → v1

- 10% → v2

✓ 점진 확대:

- 90/10 $\Rightarrow$ 70/30

$\Rightarrow$ 50/50 $\Rightarrow$ 0/100

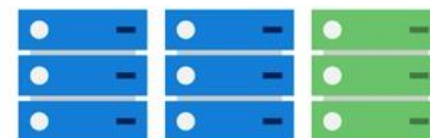
✓ 목적

- 장애 최소화
- 실제 트래픽 검증
- 성능 비교
- 빠른 롤백

State 0



State 1



State 2

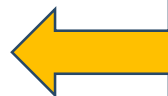


Final State



Canary 라우팅

```
http:  
- route:  
  - destination:  
    host: order-service  
    subset: v1  
    weight: 90  
  
  - destination:  
    host: order-service  
    subset: v2  
    weight: 10
```



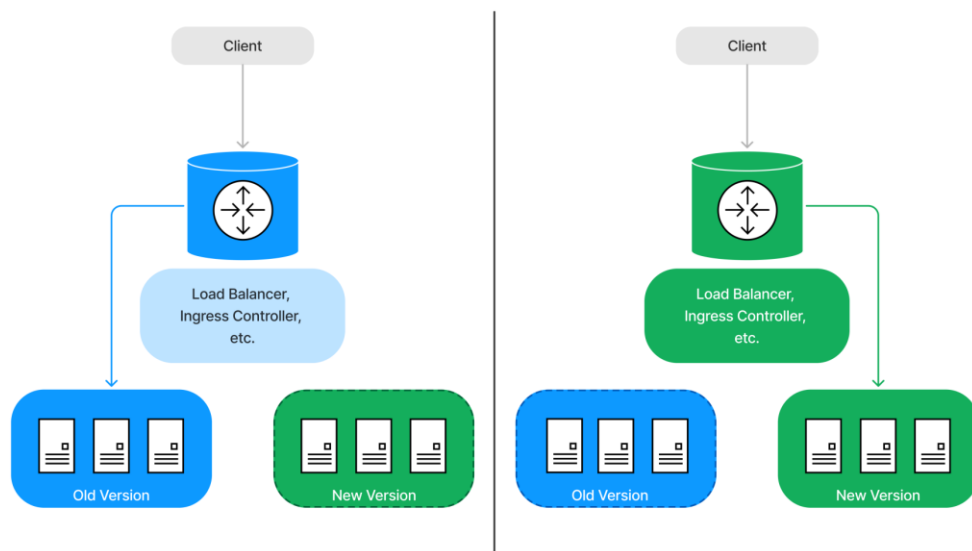
Istio를 활용한 사례

Blue/ Green 라우팅

- ✓ 두 개의 완전한 운영 환경 유지
- ✓ 구조
 - Blue = 현재 운영
 - Green = 신규 버전
- ✓ 전환 순간: Blue → Green
- ✓ 장점
 - 즉시 롤백 가능
 - 다시 Blue로 전환
 - 다운타임 거의 없음
 - 운영 환경에서 사전 검증 가능

✓ 단점

- 비용 증가: 운영 환경 2세트 필요
- DB 호환성 문제: 스키마 변경 시 어려움  DB 공유



Canary Blue/ Green 라우팅 비교

항목	Canary	Blue-Green
트래픽 분산	일부만 신규	전체 전환
위험도	매우 낮음	순간적
롤백	쉬움	매우 쉬움
운영 복잡도	높음	상대적으로 낮음
인프라 비용	상대적 적음	높음
실사용 검증	매우 좋음	제한적
배포 속도	점진적	즉시

2. 인증 / 인가 (Authentication & Authorization)

- ✓ JWT 토큰 검증
- ✓ OAuth2 / OpenID Connect 연동
- ✓ API Key 기반 인증
- ✓ Role 기반 접근 제어 (RBAC)
- ✓ Token Relay (Gateway → 내부 서비스 전달)

JWT 토큰 검증

- ✓ JWT(JSON Web Token)는 사용자 인증 정보를 담은 서명된 토큰
- ✓ 구조: Header, Payload, Signature 로 구성
- ✓ JWT 인증 흐름
 - 사용자 로그인 ⇒ 인증 서버가 JWT 발급 ⇒ 클라이언트가 JWT 저장 ⇒ API 호출 시 Authorization Header 전송 ⇒ Gateway / Service가 JWT 검증
- ✓ JWT 검증 과정
 - ① Signature 검증: 토큰이 변조되었는가?
 - ② 만료 시간(exp) 확인: 토큰이 만료되었는가?
 - ③ issuer 확인: 신뢰 가능한 인증 서버인가?
 - ④ audience 확인: 이 서비스용 토큰인가?

OAuth2 연동

- ✓ OAuth2는 “인증”이 아니라 “권한 위임” 프레임워크
 - 사용자가 비밀번호를 직접 공유하지 않고, 외부 서비스 권한을 위임
 - 예: "구글 로그인", "카카오 로그인"
- ✓ 구성 요소

구성 요소	역할
Resource Owner	사용자
Client	애플리케이션
Authorization Server	인증 서버
Resource Server	API 서버

OAuth2 연동

✓ 프로세스

- 사용자 ⇒ Google Login ⇒ Authorization Server ⇒ Access Token 발급 ⇒ Client가 API 호출

✓ Authorization Code Flow: 가장 일반적.

- 브라우저 로그인 후 인증 코드 발급하고 Access Token 교환.
- SPA + Backend + 모바일에서 널리 사용

✓ 특징

특징	설명
외부 로그인 가능	Google/Kakao/GitHub
SSO 가능	Single Sign-On
토큰 기반	Stateless
권한 위임	API 접근 제어

OpenID Connect (OIDC)

- ✓ OIDC는 OAuth2 위에 “인증(Authentication)” 기능을 추가한 것
 - OAuth2 + 사용자 인증 표준

추가 요소	목적
ID Token	사용자 신원
UserInfo Endpoint	표준 사용자 정보
subject(sub)	사용자 고유 ID
issuer(iss)	인증서버 식별
audience(aud)	토큰 대상
nonce	replay 방지
auth_time	로그인 시간

OpenID Connect (OIDC)

- ✓ OIDC는 JWT 형태의 ID Token 제공
- ✓ OIDC 로그인 흐름
 - 사용자 로그인 $\Rightarrow$ Authorization Server $\Rightarrow$ Access Token + ID Token 발급 $\Rightarrow$ Client 인증 완료
- ✓ 대표 OIDC 제공자
 - Google, Keycloak, Auth0, Okta

항목	OAuth2	OIDC
목적	권한 위임	사용자 인증
사용자 정보	표준 없음	ID Token 제공
로그인 기능	제한적	공식 지원

API Key 기반 인증

- ✓ 가장 단순한 인증 방식
- ✓ 구조
 - 클라이언트가 Key 전달
X-API-KEY: abc123
 - Authorization: ApiKey abc123
- ✓ API Key 조회: 유효 여부 확인 ⇒ 요청 허용
- ✓ 장점
 - 구현 쉽고 빠름
- ✓ 단점
 - 누가 호출했는지 알기 어렵고, 키 유출 가능

Role 기반 접근 제어 (RBAC)

- ✓ 사용자 역할(Role)에 따라 접근 제한
 - USER, ADMIN, MANAGER, OPERATOR
- ✓ JWT와 결합

```
{  
  "sub": "user1",  
  "roles": ["ADMIN", "USER"]  
}
```

API	권한
GET /products	USER
POST /products	ADMIN
DELETE /users	SUPER_ADMIN

Token Relay

- ✓ Gateway가 받은 JWT를 내부 서비스로 그대로 전달하는 방식
 - Client $\Rightarrow$ API Gateway $\Rightarrow$ Order Service $\Rightarrow$ Inventory Service
- ✓ Gateway만 인증하면 내부 서비스는 사용자 정보 모름
- ✓ 방식: Authorization Header 유지
 - Authorization: Bearer eyJ...
 - Gateway가 그대로 forward

3. 요청/응답 변환 (Transformation)

- ✓ Request Header 추가/삭제
- ✓ Response Header 가공
- ✓ Body 변환 (JSON 구조 변경)
- ✓ API Aggregation (여러 서비스 응답 통합)

Request Header 추가/삭제

- ✓ API Gateway가 요청(Request)을 내부 서비스로 전달하기 전에 HTTP Header를 수정하는 기능
- ✓ Header 추가 예:
 - X-Request-Id: abc123
 - X-User-Id: user1
 - X-Correlation-Id: order578
- ✓ 추가 이유
 - 사용자 정보 전달. 분산 추적. 내부 인증
- ✓ 삭제 이유
 - 민감 정보 제거
 - Header 충돌 방지

Response Header 가공

- ✓ 응답(Response) Header를 Gateway가 수정하는 기능 (주로 보안 이슈)
- ✓ 보안 Header 추가
 - X-Frame-Options: DENY
 - X-XSS-Protection: 1; mode=block
- ✓ 브라우저 공격 방지
- ✓ CORS 처리
 - 예: Access-Control-Allow-Origin: https://frontend.com
- ✓ 캐시 정책
- ✓ Header 제거
 - 내부 서버 정보 숨김

API Aggregation

- ✓ 여러 내부 서비스 응답을 하나로 합치는 것
- ✓ 문제 상황: 클라이언트가 주문 조회 화면 구성하려면, 주문 서비스, 사용자 서비스, 상품 서비스, 배송 서비스 모두 호출 필요
 - 호출 많음, 네트워크 증가, 모바일 비효율, 클라이언트 복잡
- ✓ Client ⇒ API Gateway / BFF ⇒ 내부 서비스 여러 개 호출 ⇒ 응답 합쳐서 반환

위치	설명
API Gateway	간단 Aggregation
BFF(Backend For Frontend)	UI 맞춤 Aggregation
GraphQL	Query 기반 Aggregation
Dedicated Aggregator Service	복잡한 조합 처리

4. Rate Limiting / Throttling

- ✓ IP 기반 요청 제한
- ✓ 사용자/토큰 기반 제한
- ✓ Burst Control (순간 트래픽 제어)
- ✓ Redis 기반 분산 Rate Limit

IP 기반 요청 제한

- ✓ 클라이언트 IP 기준으로 제한
- ✓ 예시
 - 192.168.0.1 → 1분당 100회 허용
 - 100회 초과 시:
HTTP 429 Too Many Requests
- ✓ 동작 구조
 - 요청 수 카운트 ⇒ 제한 초과 여부 확인 ⇒ 허용 or 차단

IP 기반 요청 제한

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: delivery-ingress
  namespace: default
  annotations:
    nginx.ingress.kubernetes.io/limit-rpm: "100"
    nginx.ingress.kubernetes.io/configuration-snippet: |
      limit_req_status 429;
spec:
  ingressClassName: nginx
  rules:
    - host: delivery.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: delivery-keycloak
                port:
                  number: 80
```

사용자/토큰 기반 제한

- ✓ JWT/User/API Key 기준으로 제한
- ✓ 예시
 - userId=user1 → 분당 1000회
 - API Key abc123 → 초당 50회
- ✓ 장점
 - 사용자 단위 제어
 - 멀티테넌트 지원
 - API 상품화 가능

Burst Control

- ✓ 순간적인 트래픽 폭증을 완화하는 기능
 - 평균 제한은 유지하면서 잠깐의 급증은 허용
- ✓ Burst 적용 사례
 - 평균 100 req/minute
 - Burst 500 허용
- ✓ Token Bucket 알고리즘: 대표적
 - Bucket 안에 token 존재
 - 요청 시 token 소비
 - 시간 지나면 token 재충전

```
nginx.ingress.kubernetes.io/limit-rpm: "100"  
nginx.ingress.kubernetes.io/limit-burst-  
multiplier: "5"
```

Redis 기반 분산 Rate Limit

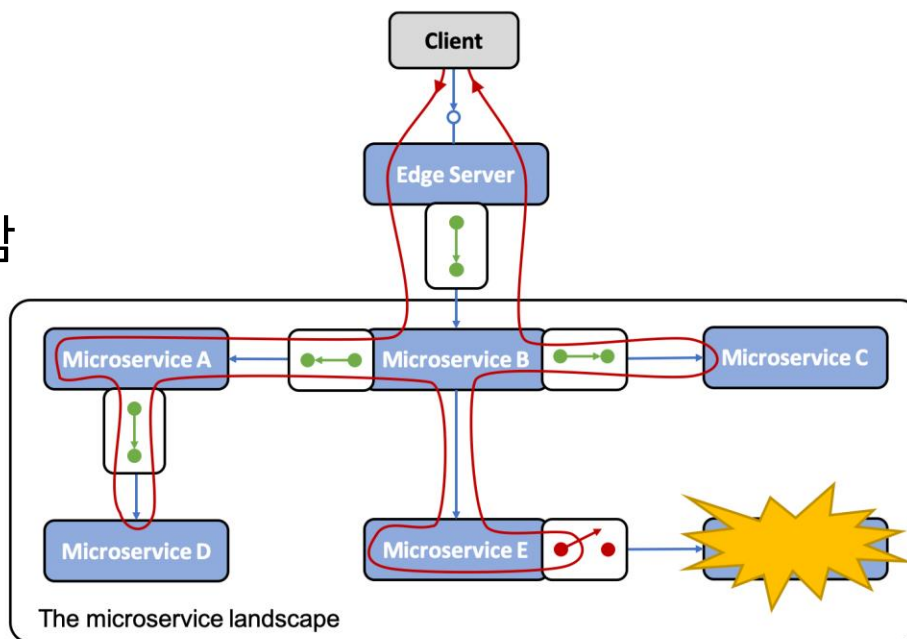
- ✓ MSA/Kubernetes에서는 서버가 여러 대이며 따라서 서버별 카운터에 따른 불일치 발생
- ✓ 예시:
 - Gateway A → 10회 허용, Gateway B → 10회 허용
 - 결과: 실제로는 20회 허용됨
- ✓ 해결방안
 - Redis 통해 공통 카운터 관리

5. Circuit Breaker (장애 격리)

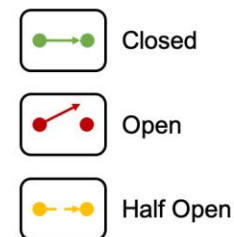
- ✓ 특정 서비스 실패 시 호출 차단
- ✓ Fail Fast 전략
- ✓ Fallback 응답 제공
- ✓ Resilience4j 연계

특정 서비스 실패 시 호출 차단

- ✓ Cascading Failure 방지
- ✓ Circuit Breaker 상태
 - CLOSED: 정상 상태
- ✓ OPEN:
 - 장애 상태
 - 요청 즉시 실패
 - 실제 서비스 호출 안 함
- ✓ HALF-OPEN
 - 복구 확인 상태
 - 일부 요청만 테스트



Circuit Breaker states



Fail Fast 전략

- ✓ 문제가 있는 서비스를 오래 기다리지 말고 즉시 실패시키는 전략
- ✓ 문제 상황
 - Service timeout 30초 $\Rightarrow$ Thread 점유 $\Rightarrow$ Connection 점유 $\Rightarrow$ 사용자 응답 지연발생
 - 100ms 안에 실패 처리
- ✓ Timeout과 함께 사용
 - Timeout + Circuit Breaker
 - 예: 2초 timeout, 5회 실패 $\Rightarrow$ Circuit OPEN

Fallback 응답 제공

- ✓ 실패 시 대체 응답 제공
 - 예: 추천 상품 조회 실패 시 기본 인기 상품 반환
 - Client $\Rightarrow$ Product Service $\Rightarrow$ Recommendation Service (장애)
 $\Rightarrow$ Fallback: Popular Products 반환
- ✓ Fallback 종류
 - Static Fallback : 고정 응답
 - Cached Fallback : 캐시 데이터 반환. Redis Cache 사용
 - Default Business Response: 기본 비즈니스 처리
예:배송비 계산 실패 $\rightarrow$ 기본 배송비 사용
- ✓ Fallback은 장애 숨김 기술. 로그, 모니터링, 장애 탐지 필수

Resilience4j 연계

- ✓ Java/Spring Boot에서 가장 대표적인 Fault Tolerance 라이브러리

기능	설명
CircuitBreaker	장애 차단
Retry	재시도
RateLimiter	요청 제한
Bulkhead	격리
Timeout	시간 제한
Cache	결과 캐싱

Resilience4j 연계

```
@CircuitBreaker(  
    name = "paymentService",  
    fallbackMethod = "fallback"  
)  
public Payment getPayment() {  
    return paymentClient.call();  
}
```

```
public Payment fallback(Exception e) {  
    return new Payment("temporary unavailable");  
}
```

```
resilience4j:  
  circuitbreaker:  
    instances:  
      paymentService:  
        failureRateThreshold: 50  
        waitDurationInOpenState: 10s  
        slidingWindowSize: 10
```

- 실패율이 50% 넘으면 Circuit OPEN
- Circuit Open 상태 유지 시간
- 최근 10개 요청 기준으로 실패율 계산

7. Retry / Timeout 제어

- ✓ 자동 재시도 정책
- ✓ Timeout 설정 (서비스별)
- ✓ Exponential Backoff: retry 시간 간격을 2^n 형태로 늘려서 retry 횟수 조정
- ✓ 장애 전파 방지

8. 로깅 / 모니터링 / 트레이싱

- ✓ 요청/응답 로그 수집
- ✓ Correlation ID 생성 및 전달
- ✓ Distributed Tracing (Zipkin, Jaeger)
- ✓ Metrics 수집 (Prometheus)

9. 보안 (Security)

- ✓ SSL/TLS Termination
- ✓ CORS 정책 적용 (CORS: Cross Origin Resource Sharing)
- ✓ IP Whitelisting / Blacklisting
- ✓ WAF 연동 가능

10. 로깅 / 모니터링 / 트레이싱

- ✓ 요청/응답 로그 수집
- ✓ Correlation ID 생성 및 전달
- ✓ Distributed Tracing (Zipkin, Jaeger)
- ✓ Metrics 수집 (Prometheus)

기술	유형	핵심 특징	장점	단점	대표 활용처
NGINX / NGINX Ingress Controller	Reverse Proxy / Ingress	가장 널리 사용되는 L7 프록시	성능 우수, 설정 단순, Kubernetes 표준적	API 관리 기능은 상대적으로 약함	Kubernetes Ingress, 단순 API Gateway
Kong	API Gateway	NGINX 기반 API Gateway	Plugin 풍부, OAuth2/OIDC 강력, 관리 쉬움	대규모에서는 DB/Plugin 관리 복잡	SaaS, MSA API Gateway
Apache APISIX	API Gateway	etcd 기반 동적 Gateway	성능 우수, 실시간 설정 변경, Plugin 강력	생태계는 Kong보다 작음	Cloud Native API Gateway
Spring Cloud Gateway	Java Reactive Gateway	Spring WebFlux 기반	Spring 생태계 통합 우수	Java 메모리 사용량 큼	Spring Boot MSA
Istio IngressGateway	Service Mesh Edge Gateway	Envoy 기반 Mesh Gateway	mTLS, Traffic Split, Canary 강력	운영 난이도 높음	대규모 Kubernetes MSA
Traefik	Cloud Native Proxy	Docker/K8s 친화적 자동 설정	매우 쉬운 설정	대기업급 정책 기능은 약함	소규모 Kubernetes

기술	유형	핵심 특징	장점	단점	대표 활용처
HAProxy	L4/L7 Load Balancer	고성능 TCP/HTTP Proxy	매우 빠름, 안정성 높음	관리 UI/API 상대적 약함	금융/고성능 시스템
Envoy	Cloud Native Proxy	CNCF 표준 프록시	xDS 기반 동적 설정, Mesh 핵심	직접 설정은 복잡	Istio/App Mesh 기반
Amazon API Gateway	Managed Gateway	AWS 완전관리형	서버 운영 불필요	벤더 종속성, 비용 증가	AWS Serverless
Apigee	Enterprise API Management	Google Enterprise API 관리	분석/정책/Portal 강력	비쌈, 무거움	대기업 API 플랫폼
WSO2 API Manager	Enterprise API Management	오픈소스 기반 Enterprise Gateway	IAM/OAuth 강력	설정 복잡	공공/금융 API
Tyk	API Gateway	Go 기반 Gateway	가볍고 빠름	생태계 상대적 작음	중소규모 API 플랫폼

 특징

-  역할

-  장점

- 설치 쉬움 (kubectl apply)
- 문서/레퍼런스 많음



Let's get started!

Deploying Nginx Ingress Controller to a Kubernetes cluster

Kong Ingress Controller

✓ 특징

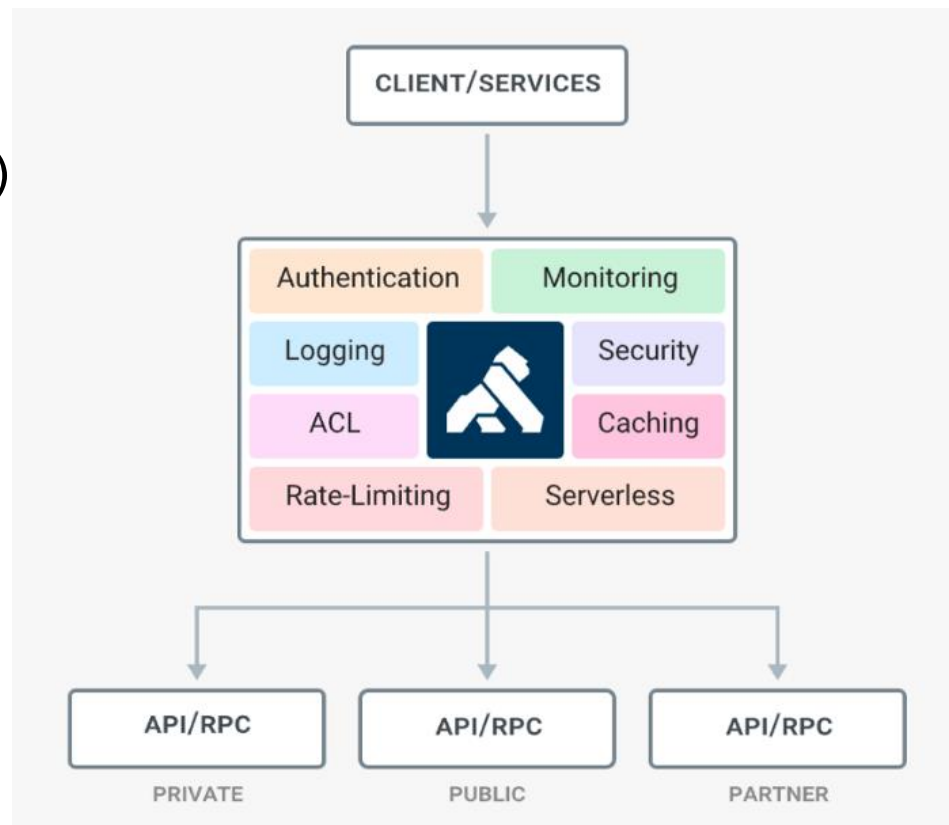
- API Gateway 전용 제품
- Plugin 기반 (인증, 로깅, rate limit 등)

✓ 장점

- API 관리 기능 강력
- DevPortal, API Key, 정책 관리 가능
- 기업용 API 플랫폼

✓ 활용

- 단순 라우팅이 아니라,
- API 관리 플랫폼 필요할 때



MSA 고급 구조 (Service Mesh): Istio + Gateway

✓ 특징

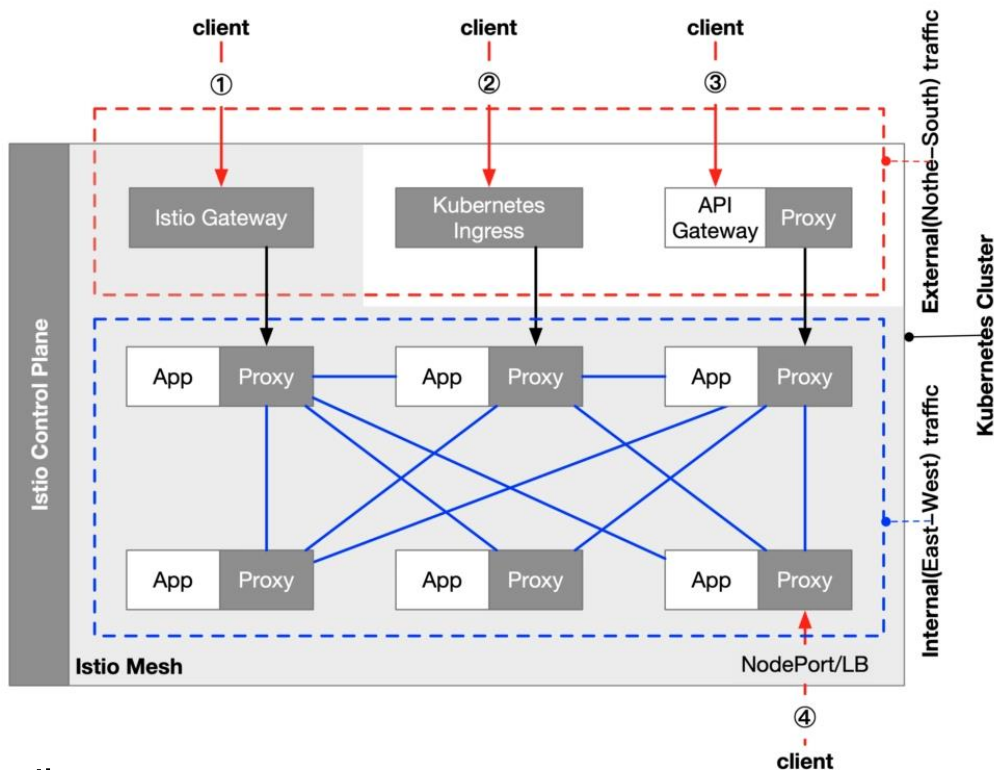
- Envoy Proxy 기반
- Gateway + Sidecar 구조

✓ 기능

- 트래픽 제어 (A/B 테스트, Canary)
- mTLS 보안서비스 간 통신 제어

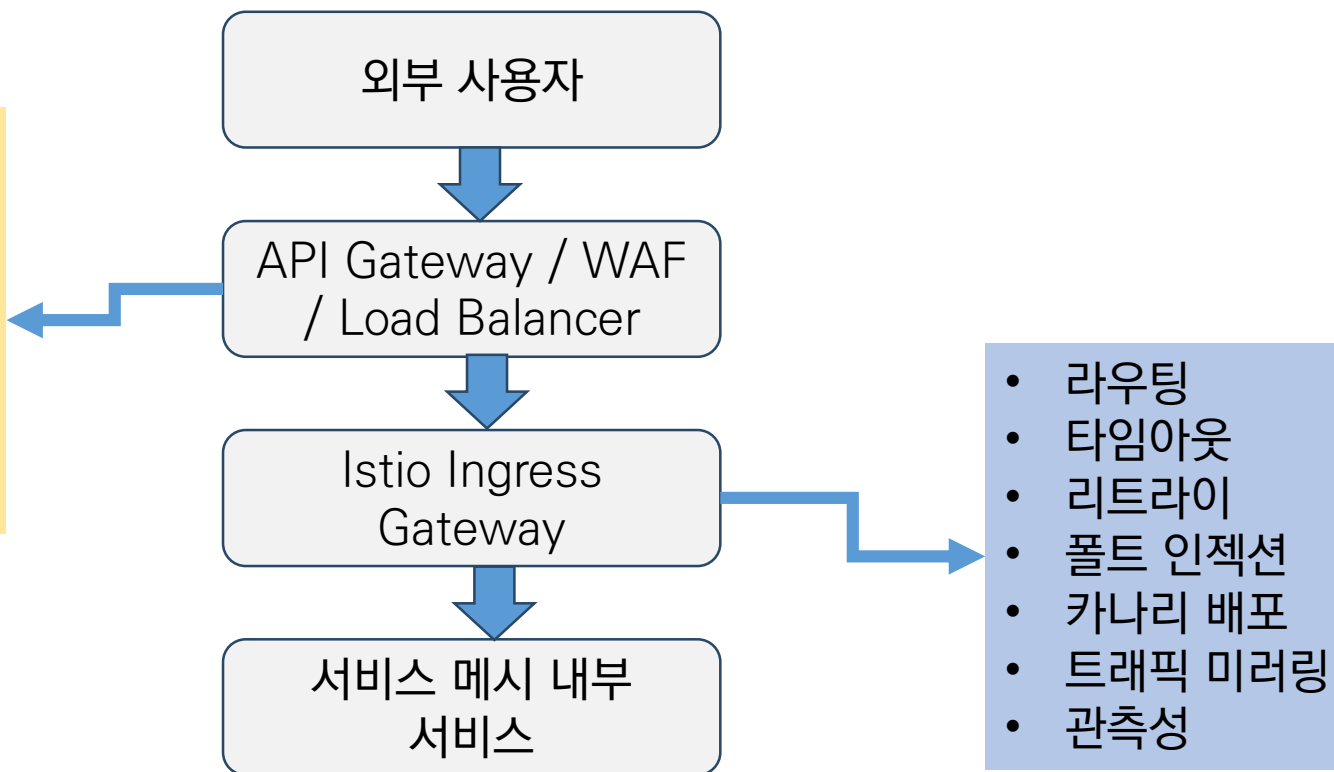
✓ 언제 쓰나?

- 대규모 MSA
- 트래픽 제어 / observability 중요할 때



실무적 관점

- API 인증,
- API Key,
- Rate Limit,
- 개발자 포털,
- 과금,
- 외부 고객 API 관리



장점

- ✓ 공통 기능 중앙화
- ✓ 보안 강화
- ✓ 서비스 독립성 향상
- ✓ 클라이언트 단순화

단점

- ✓ 단일 장애 지점 (SPOF)
- ✓ 추가 네트워크 홉
- ✓ 성능 병목 가능성
- ✓ 운영 복잡도 증가

고려 사항

- ✓ Gateway 이중화 (HA)
- ✓ 인증 방식 선택
- ✓ Rate Limiting 전략
- ✓ 장애 대응 (Circuit Breaker)
- ✓ 로그/모니터링 체계

주의 사항

- ✓ Gateway에 과도한 로직 금지
- ✓ 비즈니스 로직은 서비스에 위치
- ✓ 병목 지점 관리 필요
- ✓ timeout / retry 정책 필수

Ingress 설치하기

- ✓ `kubectl apply -f`
`https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/cloud/deploy.yaml`

설치 확인

- ✓ `kubectl get pods -n ingress-nginx`
- ✓ `kubectl get svc -n ingress-nginx`

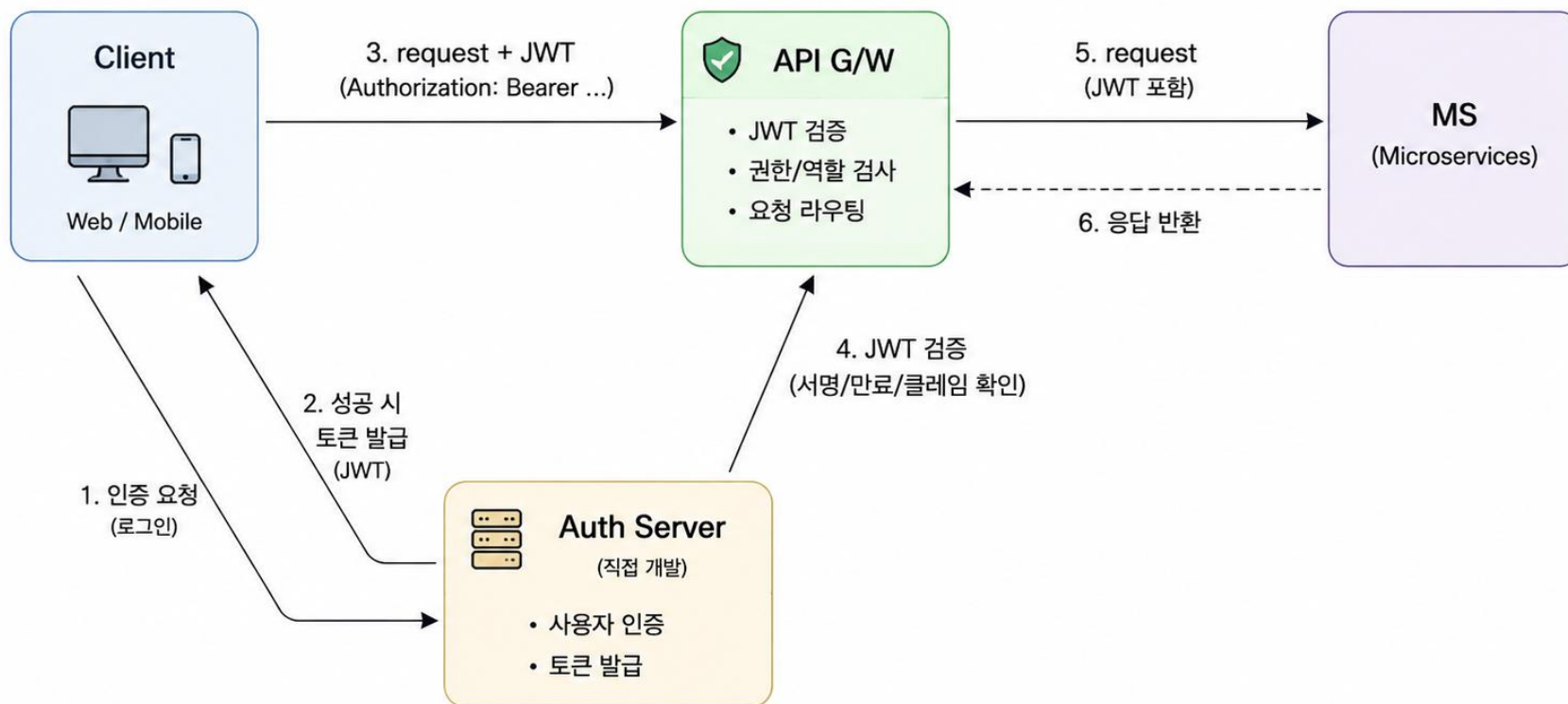
삭제하기

- ✓ `kubectl delete namespace ingress-nginx`

Ingress yaml 파일 작성: 일반적 상황

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: stockconfirm-ingress
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "false"
spec:
  ingressClassName: nginx
  rules:
    - host: localhost
      http:
        paths:
          - path: /stock
            pathType: Prefix
            backend:
              service:
                name: stockconfirm-service
                port:
                  number: 80
```


Ingress, JWT 실습



1. 인증 요청 : 클라이언트가 Auth Server로 로그인 요청
2. 토큰 발급 : 인증 성공 시 Auth Server가 JWT 토큰 발급
3. 요청 + JWT : 클라이언트가 API Gateway로 요청과 함께 JWT 전송

4. JWT 검증 : API Gateway가 JWT의 유효성 및 권한 검증
5. 요청 전달 : 검증 통과 시 해당 MS로 요청 전달
6. 응답 반환 : MS가 처리 후 API Gateway를 거쳐 클라이언트로 응답

Ingress yaml 파일 작성: 일반적 상황

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: stockconfirm-ingress
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "false"
spec:
  ingressClassName: nginx
  rules:
    - host: localhost
      http:
        paths:
          - path: /stock
            pathType: Prefix
            backend:
              service:
                name: stockconfirm-service
                port:
                  number: 80
```

Keycloak을 사용한 인증 및 OAuth2

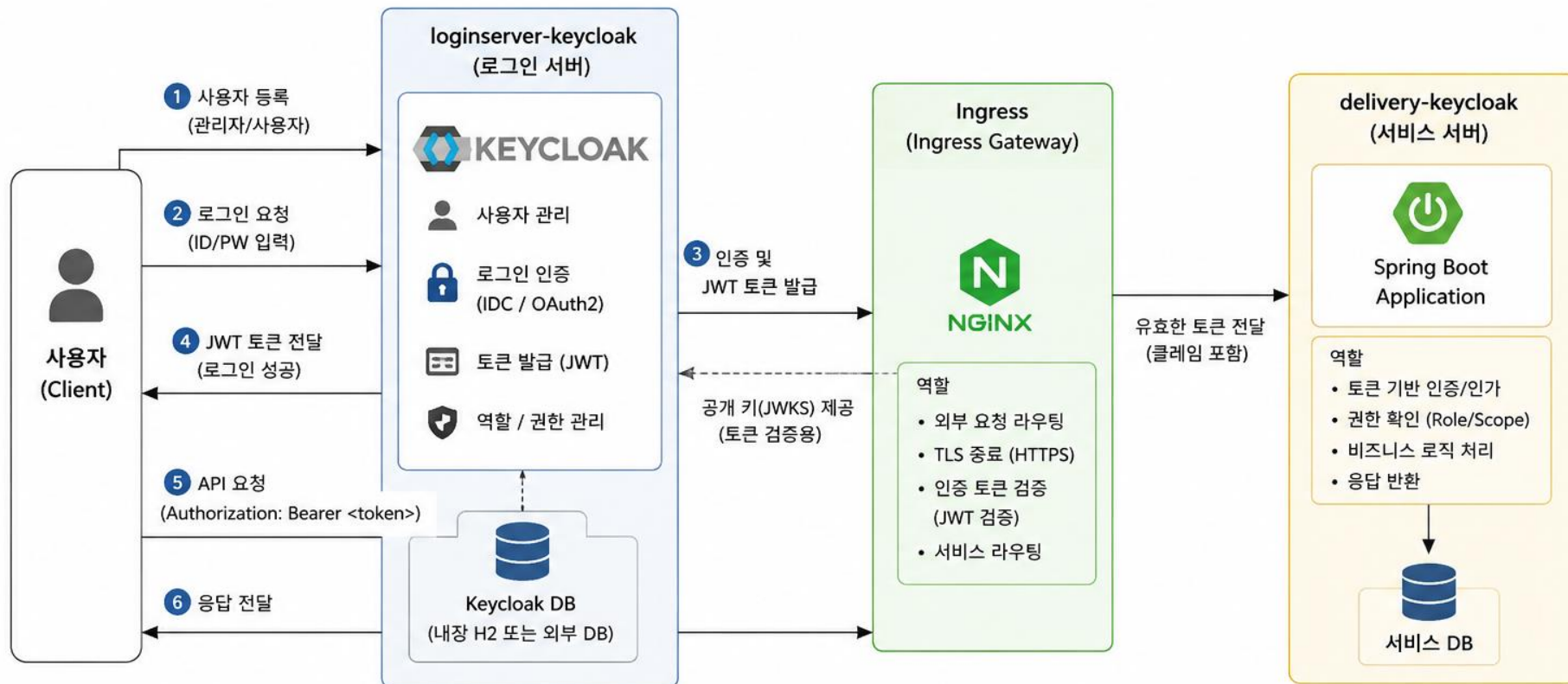
- ✓ 매우 널리 사용되는 오픈소스 IAM(Identity and Access Management) 솔루션
- ✓ 사용자 인증(Authentication)과 권한 관리(Authorization)를 중앙에서 통합 관리하는 IAM 서버
 - OIDC / OAuth2 / SAML 표준 지원
 - SSO(Single Sign-On)
 - JWT 발급
 - 소셜 로그인 연동
 - RBAC(Role 기반 권한)
 - 사용자/그룹 관리 UI 제공
 - Spring Security와 연동 쉬움
 - Kubernetes/Ingress/Istio와 결합성

KeyCloak을 사용한 인증 절차

1. Kubernetes에 Keycloak 설치: id admin, password admin123
2. Keycloak 외부 접속용 Ingress 생성
3. realm 생성: cretec
4. client 생성: loginserver-keycloak
5. user 생성
6. token 발급
7. delivery-service에 oauth2-resource-server 설정
8. Ingress로 delivery 호출

전체 흐름

1. 사용자 등록 → 2. 로그인 인증 요청 → 3. 인증 및 토큰 발급 → 4. 토큰 전달 → 5. 인그레스를 통해 서비스 접근 → 6. 서비스 응답



구성 요소 설명

- loginserver-keycloak : 사용자 등록, 로그인 인증, 토큰(JWT) 발급
- Ingress : 외부 요청 진입점, JWT 토큰 검증 후 내부 서비스로 라우팅
- delivery-keycloak : 보호된 자원(API) 제공, 토큰 기반 인가 처리
- 사용자 : 로그인 후 발급받은 토큰으로 API 호출

토큰 검증 흐름

1. Keycloak이 JWT 토큰 발급 (서명 포함)
2. Ingress가 요청의 JWT 서명 검증 (JWKS 공개키 사용)
3. 검증 성공 시 backend 서비스로 전달
4. 서비스는 필요 시 토큰의 권한/역할(claim) 확인 후 처리

Keycloak 구조

✓ Realm

- 완전히 독립된 인증 구역
- 사용자 집합
- 클라이언트 집합
- 권한 체계
- 로그인 정책을 묶는 최상위 단위

✓ Realm이 다르면:

- 사용자 공유 안됨
- 토큰 공유 안됨
- 로그인도 독립
- 권한도 독립

Realm



Keycloak 구조

- ✓ Client: Keycloak을 사용하는 애플리케이션
 - 예: Spring Boot API, Android App, Gateway
- ✓ Client ID: Client의 식별자
 - 예: client-id = loginserver-keycloak
- ✓ User: 로그인 사용자
 - Username: 사용자가 입력하는 로그인 아이디
 - User ID: Keycloak 내부 UUID (PK)
- ✓ Realm Role: Realm 전체 공통 권한
- ✓ Client Role: 특정 Client 전용 권한.
 - delivery-keycloak: DELIVERY_ADMIN, DELIVERY_USER

Keycloak 구조

- ✓ Token: Keycloak 핵심 기능
 - 로그인 성공 시, JWT Access Token을 발급
- ✓ Access Token: API 호출용 토큰
 - 예: Authorization: Bearer eyJhb...
- ✓ Refresh Token: Access Token 만료 시, 재로그인 없이 새 토큰 발급용

토큰	JWT 여부	주요 목적
Access Token	대부분 JWT	API 인증
ID Token	거의 JWT	로그인 사용자 정보
Refresh Token	JWT일 수도, 아닐 수도	토큰 재발급

Keycloak 토큰 구분

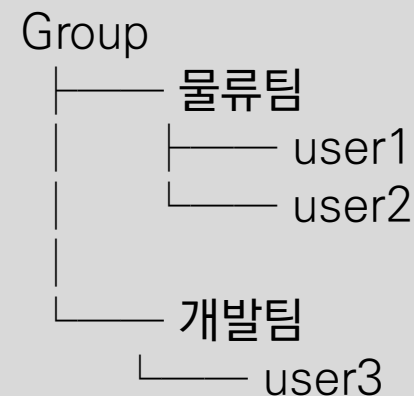
- ✓ Authentication: 사용자가 누구인지 확인. 즉 로그인
- ✓ Authorization: “무엇을 할 수 있는지 확인”
 - 예: ADMIN만 삭제 가능, USER는 조회만 가능
- ✓ Required Actions: 사용자 최초 로그인 시 강제 작업
 - 예: Verify Email, Update Password, Configure OTP, Update Profile

Keycloak 구조

- ✓ Identity Provider (IDP) : 외부 로그인 연동
 - 예: Google, GitHub, Naver, Kakao
- ✓ Federation: 외부 사용자 저장소 연동
 - 예: LDAP, Active Directory
 - 회사 계정 그대로 로그인 가능하게 함
- ✓ OIDC: Keycloak 기본 핵심 프로토콜.
 - OpenID Connect, OAuth2 기반 로그인 표준
- ✓ OAuth2: 권한 위임 표준
 - 예: Google 로그인, GitHub 로그인
- ✓ Service Account: 사용자 없이 서비스끼리 인증할 때 사용
 - delivery-service ↔ inventory-service

Keycloak 구조

- ✓ Group: 사용자 집합(조직/부서/카테고리)
 - 사용자들을 논리적으로 묶는 기능
 - 권한(Role)을 개별 사용자마다 주면 관리가 어렵기 때문
- ✓ Session : 현재 로그인 상태
 - 사용자가 로그인 성공 후 유지되는 인증 상태
 - Keycloak Session: 로그인 사용자, 로그인 시간, IP, Client, Refresh Token 상태



Keycloak 구조

- ✓ Event :Keycloak 내부에서 발생한 보안/인증 활동 기록
 - 감사(Audit Log)와 유사
 - 로그인 실패 추적, 비밀번호 변경 감사, 이상 접근 탐지, 짧은 시간 다중 로그인 실패 추적
 - 예: LOGIN, LOGIN_ERROR, LOGOUT, REGISTER, UPDATE_PASSWORD, REFRESH_TOKEN

KeyCloak을 사용한 인증 및 OAuth2

- ✓ client 생성시 client auth⇒ON, Standard flow ⇒ ON, direct access ⇒ ON

Clients > Create client

Create client

Clients are applications and services that can request authentication of a user.

1 General settings

2 Capability config

3 Login settings

Client authentication  On

Authorization  Off

Authentication flow

☒ Standard flow

☒ Direct access grants

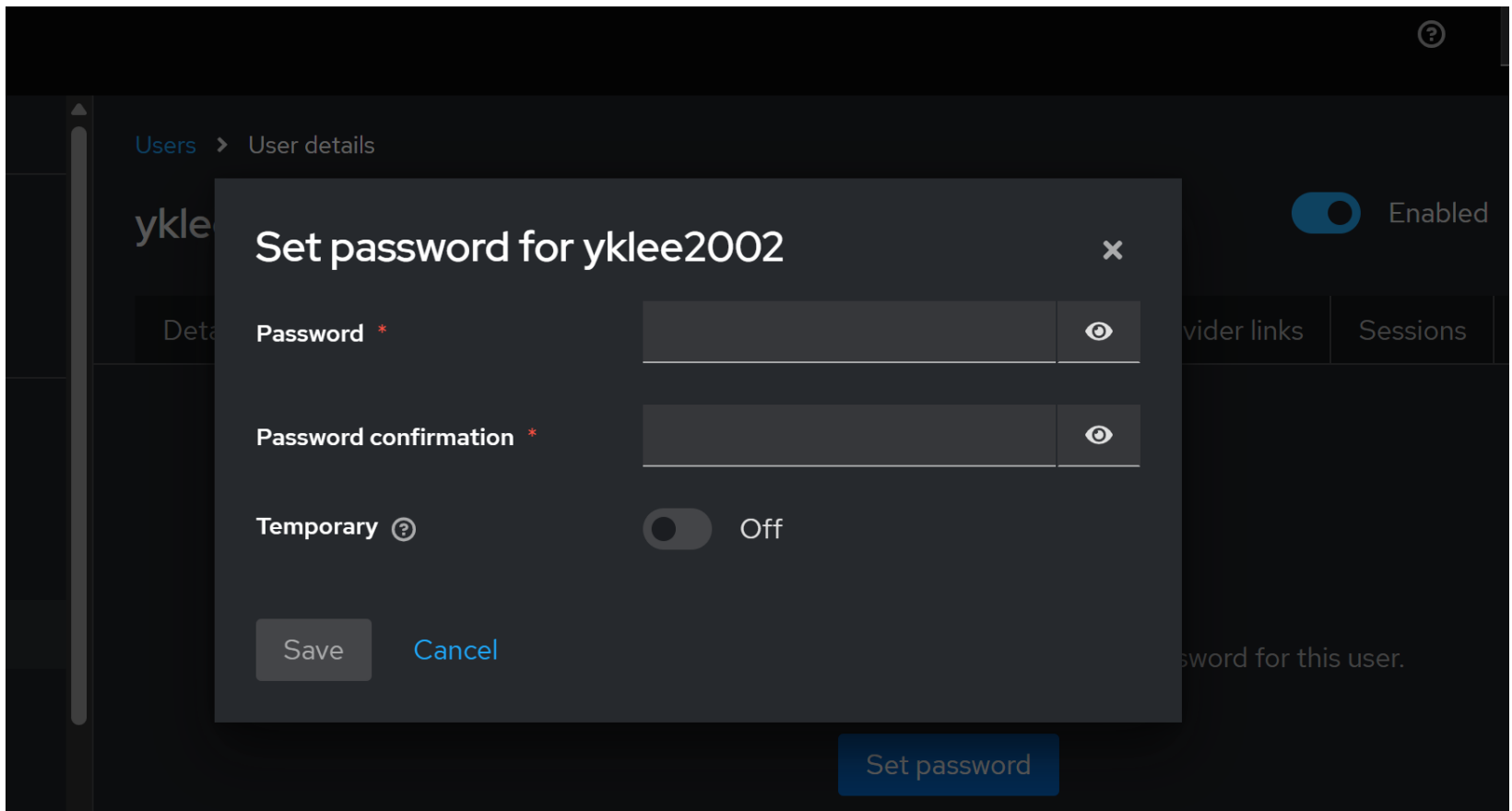
☐ Implicit flow

☐ Service accounts roles

☐ Standard Token Exchange

KeyCloak을 사용한 인증 및 OAuth2

- ✓ usre 패스워드 작성시 Temporary off 되어 있어야 함



KeyCloak을 사용한 인증 및 OAuth2

✓ delivery service 설정

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>  
</dependency>
```

✓ application.properties 설정

```
spring.security.oauth2.resourceserver.jwt.issuer-uri=  
http://keycloak.keycloak.svc.cluster.local:8080/realms/cretec
```